

FROM DESCRIPTIONS TO SOURCEABLE PARTS: CATALOG-GROUNDED RETRIEVAL FOR MECHANICAL DESIGN

Chancel Lavoie¹, Levent Burak Kara¹

¹Carnegie Mellon University, Pittsburgh, PA

ABSTRACT

AI engineering agents are beginning to translate textual requirements into mechanical part and component descriptions. To support procurement, bill-of-material generation, and CAD insertion, these descriptions must be grounded in existing catalog parts with exact part numbers. Product-data APIs can attach live links, prices, datasheets, and CAD files after identifiers are known; catalog-grounded part-number retrieval is the upstream step that makes these APIs usable by design agents. We present an open-source Model Context Protocol server for headless navigation of McMaster-Carr that converts sufficiently detailed component descriptions into matching catalog part numbers. The system renders live catalog pages, extracts visible product-table schemas, uses a language model to map requested constraints onto live field names and values, and then performs deterministic row filtering. The output is the set of catalog parts that satisfy the grounded description. In final benchmark runs, the system recovered 250/250 exact part numbers across 25 catalog categories, preserved multi-part alternatives in 50/50 underspecified cases, and returned no part in 25/25 nonexistent cases. By making part sourcing callable by agents, the method enables downstream workflows in which assemblies are crafted from off-the-shelf parts and automatically linked to product data, prices, datasheets, CAD, and bills of materials.

Keywords: AI engineering agents, design automation, part sourcing, model context protocol, catalog grounding, bill of materials, CAD retrieval

1. INTRODUCTION

AI engineering agents are increasingly used to generate mechanical part requirements, component selections, fixtures, subsystems, and assembly concepts from text. These outputs become actionable when they can be connected to existing parts that can be purchased, inspected, trusted, and inserted into CAD. Procurement, bill-of-materials generation, and CAD retrieval all depend on exact catalog identifiers, while generated requirements are typically expressed as natural-language component descriptions. This paper addresses the catalog-grounding step

between language-level engineering intent and sourceable mechanical parts.

McMaster-Carr is a widely used mechanical component catalog, and its Product Information API retrieves product information, live links, prices, images, CAD, and datasheets once a subscribed part number is known [1]. This identifier-centric interface is well suited to downstream product-data retrieval. Design agents, however, require an upstream capability: converting a textual component requirement into the catalog part numbers that satisfy it. A part-sourcing tool with this capability can connect generated component descriptions to live product data, CAD files, and bill-of-material entries.

Catalog-grounded part retrieval is challenging because industrial catalogs exhibit heterogeneous product schemas. A screw may be distinguished by material, thread size, length, drive style, package quantity, and finish. A switch may be distinguished by circuit, terminal count, maintained or momentary action, panel thickness, voltage, and terminal style. Conveyor rollers, springs, hinges, tapes, bearings, and filters each expose different discriminating fields. General retrieval therefore benefits from observing the live schema of the current catalog page and returning the complete set of rows that satisfy the description.

We present a dynamic catalog-grounding method exposed through a Model Context Protocol (MCP) server. Given a textual mechanical component description, the server returns the set of part numbers whose live catalog rows satisfy the grounded constraints. The system derives catalog search queries internally, renders live catalog pages, extracts product rows and visible column names, summarizes the live schema, aligns requested constraints with live fields and values, and filters rows programmatically. The language model serves as a schema-alignment component that produces grounded field-value matchers for the currently rendered catalog page. Once part numbers are recovered, existing product-data APIs can automatically attach live links, price information, datasheets, and CAD files to generated component or assembly descriptions.

The contribution of this work is an open-source MCP server that takes a textual mechanical component description and returns

the matching McMaster-Carr part numbers. The server packages headless rendered-page navigation, runtime product-schema extraction, language-model schema alignment, and deterministic row filtering behind a tool interface that can be called by engineering agents.

2. RELATED WORK AND BACKGROUND

Our review focuses on three bodies of related work: tool protocols for connecting language models to external systems, McMaster-Carr product-data interfaces, and prior McMaster-Carr agent tooling.

2.1. Model Context Protocol for Engineering Tools

The Model Context Protocol standardizes how language-model applications connect to external context and tools [2]. MCP servers can expose resources, prompts, and tools; tools are the primitive used when a model needs to execute an action such as querying a database, calling an API, or driving an external system [3]. Our implementation uses MCP as the integration layer so that the same part-sourcing capability can be called by any compatible agent host.

MCP is useful here because part sourcing is not simply text generation. It requires a persistent browser session, rendered-page inspection, navigation, timeouts, and a structured result contract. These actions are better exposed as tools than hidden inside a prompt.

2.2. McMaster-Carr Product Data

McMaster-Carr’s Product Information API is available to approved customers and includes requests for product information, price, image retrieval, CAD retrieval, and datasheet retrieval once a subscribed part number is known [1]. This makes part-number discovery the critical upstream step. Our system is designed to fill that step and then hand part numbers to an approved product-data pipeline.

McMaster-Carr’s web search is effective for human catalog browsing: a user can enter keywords, inspect category pages, compare table rows, apply filters, and choose a part number. An autonomous engineering agent needs a different interface. Given a product description, the agent needs a structured result containing the part numbers that satisfy the stated requirements. The proposed MCP server is therefore better suited to agentic sourcing because it converts rendered search and catalog pages into a machine-readable matching set rather than leaving the selection step to a human reader.

2.3. Existing McMaster-Carr Agent Tooling

A prior public repository, `mjbrown/mcmaster-agent`, is a useful MCP wrapper around McMaster-Carr search and product lookup [4]. If the user already has a part number, its product tool returns the product name, URL, and specification fields. If the user has a keyword query, its search tool returns candidate product hits with part numbers, names, and URLs. It also includes a URL tool for constructing product or search links. In other words, it helps an agent look up a known identifier or browse search results.

The problem addressed in this paper is one step earlier in the sourcing workflow. A design agent often starts with a product description, not a part number: for example, a detailed requirement for a screw, hinge, bearing, filter, or other catalog component. The contribution here is a tool that takes that description and returns the McMaster-Carr part numbers whose catalog rows match it. This is better suited to automated sourcing because the tool performs the selection step itself: it navigates rendered catalog pages, reads the live table schema, maps the requested constraints to the current fields, and filters the rows. The output can then be handed directly to product-data, CAD, datasheet, and BOM retrieval tools.

3. SYSTEM DESIGN

3.1. Part-Number Resolution Interface

The package exposes a Python MCP server whose central tool, `mcmaster_find_exact_part`, resolves a textual mechanical component description to the set of McMaster-Carr part numbers that match it. This is the interface studied in the paper. The input is a natural-language requirement such as a detailed description of a screw, hinge, switch, bearing, filter, or other catalog component. The output is a structured match set containing the recovered part numbers and supporting evidence from the rendered catalog rows.

This contract is intentionally narrower than general catalog browsing. The server may use internal navigation, extraction, and retry operations to reach the result, but the research object is the description-to-part-number resolver. The implementation is packaged as `mcmaster-navigator-mcp`, requires Python 3.11 or newer, and uses SeleniumBase UC mode with headless Chrome; SeleniumBase documents UC mode as an undetected-chromedriver mode for browser automation and bot-detection avoidance [5]. Model credentials are supplied at runtime rather than embedded in the package.

3.2. Headless Browser and Rendered-Page Extraction

McMaster-Carr pages are rendered web pages with product tables, category links, filters, and product detail pages. The navigator opens search URLs, auto-drills through category links when needed, and snapshots the rendered HTML. The extractor returns part numbers, product rows, row attributes, evidence text, navigable links, and schema objects containing filters, table columns, and part-number rows.

The browser is isolated by default with a temporary Chrome profile. The server serializes browser work through a single worker so browser state remains coherent across navigation calls. Browser logs are kept off MCP protocol stdout.

3.3. Dynamic Exact-Part Resolution

Figure 1 summarizes the exact-part resolver. Given a detailed component description, the first model call extracts one or more short McMaster-Carr search phrases at the product-family level. For example, a long hinge description should lead to a search phrase such as “surface-mount hinges with holes,” not a single overloaded query containing every dimension and material. The headless browser uses these phrases to reach rendered catalog pages and category tables.

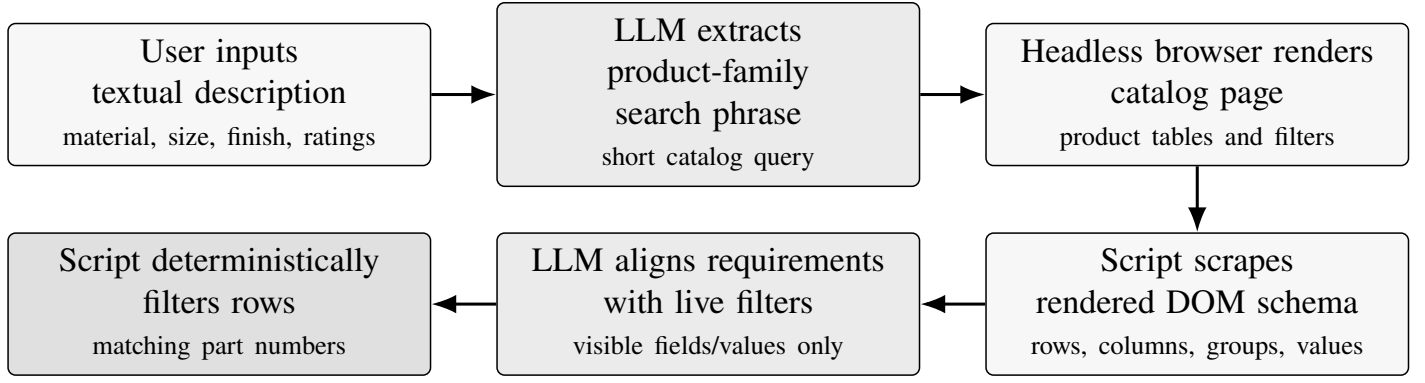


FIGURE 1: Active exact-part resolution workflow. A user-provided description is converted by the language model into a broad product-family search phrase, rendered in a headless browser, scraped into a live table and filter schema, aligned to visible fields and values, and filtered deterministically to return matching part numbers.

After the browser renders a catalog page, the extraction script snapshots the rendered DOM and parses what is actually visible: part-number rows, table columns, row groups, selected options, filter labels, and field values. This produces a runtime schema for the current product family. The second model call uses that schema to align the requested requirements with observed fields and values. It is not asked to invent a part number or rank search results. It produces grounded matchers such as material equals 316 stainless steel, range of motion equals 180 degrees, or package quantity equals 5, using the field names and values found in the rendered table.

The final selection step is programmatic. Python filtering applies the grounded matchers to the scraped rows and returns the set of rows whose values satisfy the description. If the description is specific, this set may contain one part number. If the description leaves out a discriminator, the set may contain multiple part numbers. If required constraints cannot be grounded or satisfied by visible rows, the set is empty. In benchmark runs, the schema-alignment calls used OpenAI GPT-5.4 mini, which OpenAI lists as an API model variant intended for lower latency and cost-sensitive use cases [6, 7].

This design lets the system generalize across product families because the catalog schema is discovered at runtime. There is no static list of all possible attributes for screws, springs, switches, hinges, bearings, hose fittings, raw materials, or other categories.

3.4. Assembly-Level Use Case

The core problem is part sourcing. Assembly workflows become possible by treating each off-the-shelf line item as an independent exact-part resolution problem. A component requirement is sent to `mcmaster_find_exact_part`, and the returned matching set is carried forward. Downstream systems can retrieve product data, live links, price, image, CAD, and datasheet information for returned part numbers, preserve alternatives for later selection, or revise line items whose descriptions do not match any catalog row.

This line-item independence allows parallel sourcing agents. Compatibility checking can be layered above the resolver, but the core tool intentionally solves the narrower and measurable problem of finding the catalog part numbers that match stated

requirements.

4. EXPERIMENTAL DESIGN

The evaluation measures whether the resolver returns the correct catalog match set from text rather than a plausible ranked search result. We therefore test three regimes: descriptions that should identify one known catalog part, descriptions that intentionally leave several valid alternatives, and descriptions whose required constraints cannot be satisfied.

4.1. Exact Part Recovery

The exact recovery benchmark evaluates precision when the description is sufficiently specific to identify one catalog row. We begin with real McMaster-Carr part numbers, extract the visible catalog attributes for each part from rendered product pages or table rows, transform those attributes into a textual component description, remove the part number, and ask the resolver to recover the matching part-number set. A case is counted correct only when the returned set contains exactly one part number and that part number is the held-out target.

The final exact-recovery run used 250 cases sampled across 25 catalog categories, summarized in Fig. 2.

4.2. Multi-Match Part Recovery

The multi-match benchmark evaluates whether the resolver preserves alternatives for under-specified descriptions rather than forcing a single choice. The 50 cases cover two common forms of ambiguity. In 25 controlled cases, we start from successful exact cases, collect the live schema rows around the target, and

Adhesives	Bearings	Building & Grounds	Conveying	Electrical & Lighting
Fabricating	Fastening & Joining	Filtering	Furniture & Storage	Hand Tools
Hardware	Heating & Cooling	Lab Supplies	Linear Motion	Lubricating
Measuring & Inspecting	Motors	Pipe/Tubing/Hose/Fittings	Plumbing & Janitorial	Pneumatics
Power Transmission	Pulling & Lifting	Raw Materials	Shipping	Suspending

25 catalog families: all sampled exact-recovery descriptions resolved to the held-out part number.

FIGURE 2: Exact-recovery coverage across catalog families. The grid reports the diversity of product families represented in the 250-case exact-recovery run.

remove one discriminator from the description. This creates a new description with a known matching set, usually two to four parts, computed from the same rendered catalog rows. The other 25 cases use broader descriptions such as “double sided tape,” “ball bearing,” “door hinges,” and “toggle switch,” paired with predefined expected match sets from the rendered catalog rows. These test the same behavior at larger cardinalities. A multi-match case passes only when the resolver returns the expected matching set for that case, whether the ambiguity is controlled or broad.

4.3. No-Match Part Descriptions

The no-match benchmark evaluates whether the resolver can return an empty set when the description contains requirements that no visible McMaster-Carr row satisfies. These cases start from exact descriptions and add impossible required constraints such as an unobtainable material, impossible color, or impossible size. A case passes only when the resolver returns no part numbers. This tests the failure mode most important for automated sourcing: the tool should report that no catalog match was found rather than selecting a nearby but incorrect part.

4.4. Parallel Execution for Assemblies

Assembly-level sourcing is evaluated as a parallel line-item workflow. The input is a bill-of-material style assembly description in which each off-the-shelf component is written as a separate textual requirement. Each line item is sent independently to the resolver, and the returned part-number set is associated with the corresponding assembly component. The demonstration measures whether the sourcing layer can produce a structured bill of materials whose line items can be connected to downstream product-data calls for live links, prices, datasheets, and CAD files.

5. EXPERIMENTAL RESULTS

5.1. Exact Part Recovery Results

The exact-recovery benchmark returned exactly one part number for all 250 cases, and that part number was the held-out target in every case. This corresponds to 250/250 exact recovery, or 100% accuracy. The run used an average of 7,449.96 recorded LLM tokens per case and an average wall-clock time of 71.038 s per case.

Exact: 250/250 one held-out part recovered
Multi-match: 50/50 expected set preserved
No match: 25/25 empty set returned

All completed benchmark regimes returned the expected match-set behavior.

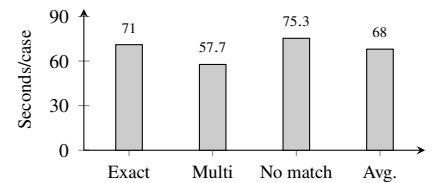
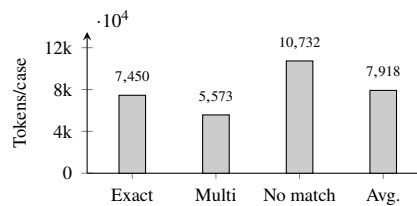


FIGURE 3: Benchmark outcome and cost summary. The resolver returns one part for sufficiently specific descriptions, preserves the expected matching set for under-specified descriptions, and returns no parts for impossible descriptions. Cost bars report mean recorded LLM tokens and mean wall-clock seconds per case.

The absence of misses across these categories is evidence that runtime schema extraction can handle substantially different product families. It is not proof of complete coverage of every McMaster-Carr page or every possible human description. The strongest defensible claim is that dynamic schema extraction and deterministic filtering can recover exact part numbers across diverse catalog families when descriptions contain sufficient identifying information.

5.2. Multi-Match Part Recovery Results

The multi-match benchmark passed all 50 under-specified cases, for a 100% multi-match pass rate. For both controlled and broad category-level descriptions, the resolver returned the expected matching set rather than collapsing the request to a single guess or returning an incomplete alternative set. These results are reported together because they test the same contract: when a

description admits more than one catalog row, the returned object should preserve the expected matching set. Across the combined multi-match set, the resolver used an average of 5,573.02 recorded LLM tokens per case and an average wall-clock time of 57.657 s per case.

5.3. No-Match Part Description Results

The no-match benchmark returned empty part-number sets for all 25 cases, for a 100% no-match pass rate. This indicates that, for the tested impossible descriptions, the resolver did not force a nearest-neighbor part when the required constraints could not be satisfied. The run used an average of 10,731.72 recorded LLM tokens per case and an average wall-clock time of 75.348 s per case.

5.4. Summary of Experimental Results

Figure 3 summarizes the completed benchmark runs. The overall reported benchmark set contains 325 cases: 250 exact-recovery cases, 50 multi-match cases, and 25 no-match cases. All 325 returned the expected match-set behavior. The cost plots compare recorded tokens and wall-clock time across benchmark categories; the average bars are unweighted means across exact, multi-match, and no-match categories.

6. CONCLUSION

This work turns a practical part-sourcing bottleneck into a measurable tool interface. A design agent can describe a required component, but a downstream product information API needs a part number. The proposed MCP server bridges that gap by headlessly navigating McMaster-Carr, extracting live product schemas, grounding textual constraints to live fields, and filtering rows deterministically. In final benchmark runs, the system recovered 250/250 exact part numbers across 25 catalog categories, preserved multi-part alternatives when requirements were under-specified, and returned no part when requirements could not be satisfied. Reliable part sourcing can let agents compose off-the-shelf components into larger designs while automatically attaching BOM entries, CAD, datasheets, prices, live links, and product data.

7. DISCUSSION

The central result is that part sourcing can be framed as catalog-grounded constraint satisfaction rather than ordinary semantic search. A keyword search for “ball bearing” or “socket head screw” may surface relevant catalog areas, but it does not decide which row satisfies a detailed engineering requirement. The proposed resolver uses search only to reach the appropriate product family. The selection step is then performed over rows, columns, groups, and values extracted from the rendered catalog page.

This distinction matters because mechanical catalogs are schema heterogeneous. The fields that identify a hinge are not the fields that identify a switch, filter, spring, bearing, or pneumatic fitting. A fixed parser would either miss many categories or gradually become a hand-coded catalog ontology. In contrast, the dynamic approach makes each page describe itself: the resolver reads the visible table and asks the model to align the description to that local schema. The model is therefore used for interpretation of field meaning, while the final part-number set is produced by deterministic row filtering.

The returned set is also part of the contribution. In sourcing, a single answer is only appropriate when the requirement actually identifies one catalog row. If the description omits shaft diameter, seal type, finish, voltage, or another discriminator, multiple rows may satisfy the stated requirement. Returning those alternatives is more faithful than forcing a best guess. Likewise, an empty set is the correct result when a required constraint is absent or contradicted by the visible catalog data. The exact, multi-match, and no-match evaluations test these three cardinalities of the same contract.

The method complements product-data APIs rather than replacing them. Once part numbers are known, downstream retrieval of prices, datasheets, CAD files, images, and live links is a conventional API task [1]. The contribution here is the upstream grounding step that makes those identifier-centric APIs usable when a design agent begins with component descriptions.

8. LIMITATIONS AND FUTURE WORK

8.1. Benchmark Descriptions

The benchmark descriptions were generated from rendered catalog rows and product pages. They are specific and often in-

clude labels such as family, group, selected option, and attribute names. This is a valid test of schema grounding and exact retrieval, but it is not the same as testing noisy human design notes. Future work should evaluate requirements written directly by designers and requirements produced by upstream design agents without access to catalog row labels.

8.2. Latency

Latency is dominated by headless browsing. The mean exact-recovery latency in the final run was 71.038 seconds per case. Parallel line-item sourcing can reduce assembly-level wall-clock time, but single-line resolution remains slow relative to a pure API call. Future work should cache rendered schemas, reuse stable category pages, and parallelize independent search phrases where doing so does not overload the catalog site.

8.3. Page Layout and Availability

The system depends on rendered website structure. If McMaster-Carr changes page layout, table markup, anti-automation behavior, site availability, or part-number presentation, extraction may require maintenance. Because the implementation is headless-only, vendor-side changes to browser detection can also temporarily prevent retrieval until the navigation layer is updated.

8.4. Extension to Other Vendors

The method is not inherently limited to McMaster-Carr. Other industrial catalogs and distributors also expose product families, filters, tables, and specification fields. Extending the system to other vendors would require vendor-specific navigation and extraction adapters, but the core pattern can remain the same: retrieve a relevant product family, extract the live schema, align textual requirements to visible fields, and filter rows. Multi-vendor sourcing is an important future direction because engineering agents often need alternatives across suppliers, inventory states, and purchasing constraints.

8.5. Product API Access

The system is not affiliated with McMaster-Carr and should be used in accordance with applicable terms and approved API access. The official product information API requires approved customer access and subscription to product information [1]. Therefore, this work should be understood as part-number retrieval and grounding; downstream enrichment with live prices, CAD, datasheets, and product information depends on the user’s authorized access to those APIs.

8.6. Assembly-Level Validation

Future work should complete a full assembly-level sourcing demonstration with compatibility checks across line items. The current line-item contract makes parallel sourcing possible, but assembly validity also depends on relationships between parts such as mating thread sizes, voltage compatibility, mounting geometry, and load ratings.

REFERENCES

- [1] McMaster-Carr. “McMaster-Carr Product Information API.” McMaster-Carr (2026). Accessed 2026-05-11, URL <https://www.mcmaster.com/help/api/>.
- [2] Model Context Protocol. “Model Context Protocol Specification.” Model Context Protocol (2024). Accessed 2026-05-11, URL <https://modelcontextprotocol.io/specification/2024-11-05/index>.
- [3] Model Context Protocol. “Tools.” Model Context Protocol (2026). Accessed 2026-05-11, URL <https://modelcontextprotocol.io/specification/draft/server/tools>.
- [4] Braun, Michael J. “McMaster-Carr MCP Server.” GitHub (2026). Accessed 2026-05-11, URL <https://github.com/mjbraun/mcmaster-agent>.
- [5] SeleniumBase. “UC Mode.” SeleniumBase (2026). Accessed 2026-05-11, URL https://seleniumbase.io/help_docs/uc_mode/.
- [6] OpenAI. “GPT-5.4 mini Model.” OpenAI (2026). Accessed 2026-05-11, URL <https://developers.openai.com/api/docs/models/gpt-5.4-mini/>.
- [7] OpenAI. “Models.” OpenAI (2026). Accessed 2026-05-11, URL <https://developers.openai.com/api/docs/models>.